Reading, Indexing, and Summarizing Dataframes

Steve Broll

Spring 2019

DataFrames

data.frame() Function

First create some numeric vectors of equal length

```
v21 <- seq(1,10,1)
v22 <- v21 ** 2
v23 <- seq(20,15.5,-.5)
length(v21) == length(v23)
```

```
## [1] TRUE
```

Use data.frame() Function to combine our 3 vectors into a dataframe

```
data.frame(v21,v22,v23) # value format
```

```
##    v21 v22  v23
## 1    1   1 20.0
## 2    2   4 19.5
## 3    3   9 19.0
## 4    4  16 18.5
## 5    5  25 18.0
## 6    6  36 17.5
## 7    7  49 17.0
## 8    8  64 16.5
## 9    9  81 16.0
## 10  10 100 15.5
```

Each column can be manually named in the data.frame() Function

```
data.frame('A' = v21,'B' = v22, 'C' = v23) # tag value format
```

```
##     A   B    C
## 1   1   1 20.0
## 2   2   4 19.5
## 3   3   9 19.0
## 4   4  16 18.5
## 5   5  25 18.0
## 6   6  36 17.5
## 7   7  49 17.0
## 8   8  64 16.5
## 9   9  81 16.0
## 10 10 100 15.5
```

Store DataFrame into variable 'df'

```
df <- data.frame('A' = v21,'B' = v22, 'C' = v23)
```

Print out full data set

```
df
```

```
##     A   B    C
## 1   1   1 20.0
## 2   2   4 19.5
## 3   3   9 19.0
## 4   4  16 18.5
## 5   5  25 18.0
## 6   6  36 17.5
## 7   7  49 17.0
## 8   8  64 16.5
## 9   9  81 16.0
## 10 10 100 15.5
```

Print out first 6 rows of DataFrame

```
head(df)
```

```
##   A  B    C
## 1 1  1 20.0
## 2 2  4 19.5
## 3 3  9 19.0
## 4 4 16 18.5
## 5 5 25 18.0
## 6 6 36 17.5
```

Print out first 3 rows of DataFrame

```
head(df, 3)
```

```
##   A B    C
## 1 1 1 20.0
## 2 2 4 19.5
## 3 3 9 19.0
```

Indexing a DataFrame

DataFrames use brackets, like vectors, but in a [row,column] format

```
df[1,3]
```

```
## [1] 20
```

If we only include one number 'n' in our index, the nth column will be selected

```
df[3]
```

```
##       C
## 1  20.0
```

```
## 2  19.5
## 3  19.0
## 4  18.5
## 5  18.0
## 6  17.5
## 7  17.0
## 8  16.5
## 9  16.0
## 10 15.5
```

If we want to include a specific set of rows in a column, we can use the ':' operator naturally

```r
df[1:3,]
```

```
##   A B    C
## 1 1 1 20.0
## 2 2 4 19.5
## 3 3 9 19.0
```

Similarly, if we want to include row values from a set of columns, we can use the ':' operator

```r
df[1,2:3]
```

```
##   B  C
## 1 1 20
```

The ':' operator can be used for rows and columns at the same time

```r
df[1:6,2:3]
```

```
##    B    C
## 1  1 20.0
## 2  4 19.5
## 3  9 19.0
## 4 16 18.5
## 5 25 18.0
## 6 36 17.5
```

We can see similar results by using the head() Function on an indexed df

```r
head(df[,2:3])
```

```
##    B    C
## 1  1 20.0
## 2  4 19.5
## 3  9 19.0
## 4 16 18.5
## 5 25 18.0
## 6 36 17.5
```

As with vectors, we can create a new df, named 'df2' here, from an indexed df

```r
df2 <- df[1:6,2:3]
df2
```

```
##    B    C
## 1  1 20.0
## 2  4 19.5
## 3  9 19.0
## 4 16 18.5
## 5 25 18.0
## 6 36 17.5
```

Select all columns with specific rows by leaving right side of comma blank

```r
df[1:5,]
```

```
##   A  B    C
## 1 1  1 20.0
## 2 2  4 19.5
## 3 3  9 19.0
## 4 4 16 18.5
## 5 5 25 18.0
```

Select all rows with specific columns by leaving left side of comma blank

```r
df[,1:2]
```

```
##     A   B
## 1   1   1
## 2   2   4
## 3   3   9
## 4   4  16
## 5   5  25
## 6   6  36
## 7   7  49
## 8   8  64
## 9   9  81
## 10 10 100
```

```r
df[,2]
```

```
##  [1]   1   4   9  16  25  36  49  64  81 100
```

Indexing DataFrames with column names

When the column name has no spaces, the expression df$A will return the vector we assigned to A (v21)

```r
df$A # will output vector
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

```r
df$A == v21
```

```
##  [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

The expression df['A'] will be similar, but not equivalent

```
df['A'] # named column
```

```
##     A
## 1   1
## 2   2
## 3   3
## 4   4
## 5   5
## 6   6
## 7   7
## 8   8
## 9   9
## 10 10
```

```
df[1,'A'] # will output single element
```

```
## [1] 1
```

```
df$A[1]
```

```
## [1] 1
```

Selecting Rows with Logical

Using a logical operator on df without a comma will print a vector of all values meeting the logical statement

```
df[df > 3]
```

```
##  [1]   4.0   5.0   6.0   7.0   8.0   9.0  10.0   4.0   9.0  16.0  25.0
## [12]  36.0  49.0  64.0  81.0 100.0  20.0  19.5  19.0  18.5  18.0  17.5
## [23]  17.0  16.5  16.0  15.5
```

Take a closer look at what's happening

```
df > 3
```

```
##           A     B    C
##  [1,] FALSE FALSE TRUE
##  [2,] FALSE  TRUE TRUE
##  [3,] FALSE  TRUE TRUE
##  [4,]  TRUE  TRUE TRUE
##  [5,]  TRUE  TRUE TRUE
##  [6,]  TRUE  TRUE TRUE
##  [7,]  TRUE  TRUE TRUE
##  [8,]  TRUE  TRUE TRUE
##  [9,]  TRUE  TRUE TRUE
## [10,]  TRUE  TRUE TRUE
```

Logical operator can be performed on a specific column or columns

```
df[,1] > 3
```

```
##  [1] FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

```r
df[df[1] > 3]
```

```
##  [1]   4.0   5.0   6.0   7.0   8.0   9.0  10.0  16.0  25.0  36.0  49.0
## [12]  64.0  81.0 100.0  18.5  18.0  17.5  17.0  16.5  16.0  15.5
```

While the above works, it is not recommended because of the vector format, readability of the code, and risk of error

```r
df[1] == df[,1]
```

```
##          A
##  [1,] TRUE
##  [2,] TRUE
##  [3,] TRUE
##  [4,] TRUE
##  [5,] TRUE
##  [6,] TRUE
##  [7,] TRUE
##  [8,] TRUE
##  [9,] TRUE
## [10,] TRUE
```

```r
df[df[,1] > 3]
```

```
## Error in `[.data.frame`(df, df[, 1] > 3): undefined columns selected
```

If we are using a logical operator on a column, using some of the alternate column notations are much more readable

```r
df[df$A > 3,]
```

```
##     A   B    C
## 4   4  16 18.5
## 5   5  25 18.0
## 6   6  36 17.5
## 7   7  49 17.0
## 8   8  64 16.5
## 9   9  81 16.0
## 10 10 100 15.5
```

Take a closer look at the logic inside the indices

```r
df$A > 3
```

```
##  [1] FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

Access different column than one used in the logical statement

```r
df[df$A > 3,'C']
```

```
## [1] 18.5 18.0 17.5 17.0 16.5 16.0 15.5
```

```r
df[df$A > 3,'C']
```

```
## [1] 18.5 18.0 17.5 17.0 16.5 16.0 15.5
```

Selecting Rows with which()

```
df[which(df$A >3),]
```

```
##     A   B    C
## 4   4  16 18.5
## 5   5  25 18.0
## 6   6  36 17.5
## 7   7  49 17.0
## 8   8  64 16.5
## 9   9  81 16.0
## 10 10 100 15.5
```

The following are equivalent:

```
df[which(df$A > 3),] == df[df$A > 3,]
```

```
##       A    B    C
## 4  TRUE TRUE TRUE
## 5  TRUE TRUE TRUE
## 6  TRUE TRUE TRUE
## 7  TRUE TRUE TRUE
## 8  TRUE TRUE TRUE
## 9  TRUE TRUE TRUE
## 10 TRUE TRUE TRUE
```

We can also use the identical function to verify they are equivalent

```
identical(df[which(df$A > 3),],  df[df$A > 3,])
```

```
## [1] TRUE
```

Example with Real Data

read.csv() Function

If file is in working directory we do not need to specify file path, just the file name

```
cereal <- read.csv('cereal.csv')
head(cereal)
```

```
##                  X Calories Protein Fat Sodium Fiber Carbohydrates Sugar
## 1       ACCheerios      110       2   2    180   1.5          10.5    10
## 2         Cheerios      110       6   2    290   2.0          17.0     1
## 3       CocoaPuffs      110       1   1    180   0.0          12.0    13
## 4     CountChocula      110       1   1    180   0.0          12.0    13
## 5    GoldenGrahams      110       1   1    280   0.0          15.0     9
## 6 HoneyNutCheerios      110       3   1    250   1.5          11.5    10
##   Potassium
## 1        70
## 2       105
## 3        55
## 4        65
## 5        45
## 6        90
```

names() Function

Change name of first column

```
names(cereal)
```

```
## [1] "X"             "Calories"      "Protein"       "Fat"
## [5] "Sodium"        "Fiber"         "Carbohydrates" "Sugar"
## [9] "Potassium"
```

```
names(cereal)[1]
```

```
## [1] "X"
```

```
names(cereal)[1] <- 'Cereal'
names(cereal)
```

```
## [1] "Cereal"        "Calories"      "Protein"       "Fat"
## [5] "Sodium"        "Fiber"         "Carbohydrates" "Sugar"
## [9] "Potassium"
```

```
head(cereal)
```

```
##             Cereal Calories Protein Fat Sodium Fiber Carbohydrates Sugar
## 1       ACCheerios      110       2   2    180   1.5          10.5    10
## 2         Cheerios      110       6   2    290   2.0          17.0     1
## 3       CocoaPuffs      110       1   1    180   0.0          12.0    13
## 4     CountChocula      110       1   1    180   0.0          12.0    13
## 5    GoldenGrahams      110       1   1    280   0.0          15.0     9
## 6 HoneyNutCheerios      110       3   1    250   1.5          11.5    10
##   Potassium
## 1        70
## 2       105
```

```
## 3        55
## 4        65
## 5        45
## 6        90
```

View() Function

```r
View(cereal) # opens new tab for data viewer
```

Indexing continued

Access first column 'X'

```r
cereal[,1]
```

```
##  [1] ACCheerios              Cheerios
##  [3] CocoaPuffs              CountChocula
##  [5] GoldenGrahams           HoneyNutCheerios
##  [7] Kix                     LuckyCharms
##  [9] MultiGrainCheerios      OatmealRaisinCrisp
## [11] RaisinNutBran           TotalCornFlakes
## [13] TotalRaisinBran         TotalWholeGrain
## [15] Trix                    Cheaties
## [17] WheatiesHoneyGold       AllBran
## [19] AppleJacks              CornFlakes
## [21] CornPops                CracklinOatBran
## [23] Crispix                 FrootLoops
## [25] FrostedFlakes           FrostedMiniWheats
## [27] FruitfulBran            JustRightCrunchyNuggets
## [29] MueslixCrispyBlend      NutNHoneyCrunch
## [31] NutriGrainAlmondRaisin  NutriGrainWheat
## [33] Product19               RaisinBran
## [35] RiceKrispies            Smacks
## [37] SpecialK                CapNCrunch
## [39] HoneyGrahamOhs          Life
## [41] PuffedRice              PuffedWheat
## [43] QuakerOatmeal
## 43 Levels: ACCheerios AllBran AppleJacks CapNCrunch Cheaties ... WheatiesHoneyGold
```

```r
cereal[,'Cereal']
```

```
##  [1] ACCheerios              Cheerios
##  [3] CocoaPuffs              CountChocula
##  [5] GoldenGrahams           HoneyNutCheerios
##  [7] Kix                     LuckyCharms
##  [9] MultiGrainCheerios      OatmealRaisinCrisp
## [11] RaisinNutBran           TotalCornFlakes
## [13] TotalRaisinBran         TotalWholeGrain
## [15] Trix                    Cheaties
## [17] WheatiesHoneyGold       AllBran
## [19] AppleJacks              CornFlakes
## [21] CornPops                CracklinOatBran
## [23] Crispix                 FrootLoops
```

```
## [25] FrostedFlakes           FrostedMiniWheats
## [27] FruitfulBran            JustRightCrunchyNuggets
## [29] MueslixCrispyBlend      NutNHoneyCrunch
## [31] NutriGrainAlmondRaisin  NutriGrainWheat
## [33] Product19               RaisinBran
## [35] RiceKrispies            Smacks
## [37] SpecialK                CapNCrunch
## [39] HoneyGrahamOhs          Life
## [41] PuffedRice              PuffedWheat
## [43] QuakerOatmeal
## 43 Levels: ACCheerios AllBran AppleJacks CapNCrunch Cheaties ... WheatiesHoneyGold
```

```
cereal$Cereal
```

```
##  [1] ACCheerios              Cheerios
##  [3] CocoaPuffs              CountChocula
##  [5] GoldenGrahams           HoneyNutCheerios
##  [7] Kix                     LuckyCharms
##  [9] MultiGrainCheerios      OatmealRaisinCrisp
## [11] RaisinNutBran           TotalCornFlakes
## [13] TotalRaisinBran         TotalWholeGrain
## [15] Trix                    Cheaties
## [17] WheatiesHoneyGold       AllBran
## [19] AppleJacks              CornFlakes
## [21] CornPops                CracklinOatBran
## [23] Crispix                 FrootLoops
## [25] FrostedFlakes           FrostedMiniWheats
## [27] FruitfulBran            JustRightCrunchyNuggets
## [29] MueslixCrispyBlend      NutNHoneyCrunch
## [31] NutriGrainAlmondRaisin  NutriGrainWheat
## [33] Product19               RaisinBran
## [35] RiceKrispies            Smacks
## [37] SpecialK                CapNCrunch
## [39] HoneyGrahamOhs          Life
## [41] PuffedRice              PuffedWheat
## [43] QuakerOatmeal
## 43 Levels: ACCheerios AllBran AppleJacks CapNCrunch Cheaties ... WheatiesHoneyGold
```

Access second column 'Calories'

```
cereal[,2]
```

```
##  [1] 110 110 110 110 110 110 110 110 100 130 100 110 140 100 110 100 110
## [18]  70 110 100 110 110 110 110 110 100 120 110 160 120 140  90 100 120
## [35] 110 110 110 120 120 100  50  50 100
```

```
cereal[,'Calories']
```

```
##  [1] 110 110 110 110 110 110 110 110 100 130 100 110 140 100 110 100 110
## [18]  70 110 100 110 110 110 110 110 100 120 110 160 120 140  90 100 120
## [35] 110 110 110 120 120 100  50  50 100
```

```
cereal$Calories
```

```
##  [1] 110 110 110 110 110 110 110 110 100 130 100 110 140 100 110 100 110
## [18]  70 110 100 110 110 110 110 110 100 120 110 160 120 140  90 100 120
## [35] 110 110 110 120 120 100  50  50 100
```

Access every column in the first row

```
cereal[1,]
```

```
##       Cereal Calories Protein Fat Sodium Fiber Carbohydrates Sugar
## 1 ACCheerios      110       2   2    180   1.5          10.5    10
##   Potassium
## 1        70
```

Access every column in the first 3 rows

```
cereal[1:3,]
```

```
##       Cereal Calories Protein Fat Sodium Fiber Carbohydrates Sugar
## 1 ACCheerios      110       2   2    180   1.5          10.5    10
## 2   Cheerios      110       6   2    290   2.0          17.0     1
## 3 CocoaPuffs      110       1   1    180   0.0          12.0    13
##   Potassium
## 1        70
## 2       105
## 3        55
```

Access the 5th - 10th cereal brand names

```
cereal[5:10,'Cereal']
```

```
## [1] GoldenGrahams      HoneyNutCheerios   Kix
## [4] LuckyCharms        MultiGrainCheerios OatmealRaisinCrisp
## 43 Levels: ACCheerios AllBran AppleJacks CapNCrunch Cheaties ... WheatiesHoneyGold
```

Access the 5th - 10th cereal brand names, calories, and protein

```
cereal[5:10,1:3]
```

```
##                Cereal Calories Protein
## 5       GoldenGrahams      110       1
## 6    HoneyNutCheerios      110       3
## 7                 Kix      110       2
## 8         LuckyCharms      110       2
## 9  MultiGrainCheerios      100       2
## 10 OatmealRaisinCrisp      130       3
```

Access every column for the following rows: 2,5,19,34,35

```
cereal[c(2,5,19,34,35),]
```

```
##           Cereal Calories Protein Fat Sodium Fiber Carbohydrates Sugar
## 2       Cheerios      110       6   2    290     2            17     1
## 5  GoldenGrahams      110       1   1    280     0            15     9
## 19    AppleJacks      110       2   0    125     1            11    14
## 34    RaisinBran      120       3   1    210     5            14    12
## 35  RiceKrispies      110       2   0    290     0            22     3
##    Potassium
## 2        105
## 5         45
```

```
## 19        30
## 34       240
## 35        35
```

Access all but the first column

```r
cereal[,-1]
```

```
##    Calories Protein Fat Sodium Fiber Carbohydrates Sugar Potassium
## 1       110       2   2    180   1.5          10.5    10        70
## 2       110       6   2    290   2.0          17.0     1       105
## 3       110       1   1    180   0.0          12.0    13        55
## 4       110       1   1    180   0.0          12.0    13        65
## 5       110       1   1    280   0.0          15.0     9        45
## 6       110       3   1    250   1.5          11.5    10        90
## 7       110       2   1    260   0.0          21.0     3        40
## 8       110       2   1    180   0.0          12.0    12        55
## 9       100       2   1    220   2.0          15.0     6        90
## 10      130       3   2    170   1.5          13.5    10       120
## 11      100       3   2    140   2.5          10.5     8       140
## 12      110       2   1    200   0.0          21.0     3        35
## 13      140       3   1    190   4.0          15.0    14       230
## 14      100       3   1    200   3.0          16.0     3       110
## 15      110       1   1    140   0.0          13.0    12        25
## 16      100       3   1    200   3.0          17.0     3       110
## 17      110       2   1    200   1.0          16.0     8        60
## 18       70       4   1    260   9.0           7.0     5       320
## 19      110       2   0    125   1.0          11.0    14        30
## 20      100       2   0    290   1.0          21.0     2        35
## 21      110       1   0     90   1.0          13.0    12        20
## 22      110       3   3    140   4.0          10.0     7       160
## 23      110       2   0    220   1.0          21.0     3        30
## 24      110       2   1    125   1.0          11.0    13        30
## 25      110       1   0    200   1.0          14.0    11        25
## 26      100       3   0      0   3.0          14.0     7       100
## 27      120       3   0    240   5.0          14.0    12       190
## 28      110       2   1    170   1.0          17.0     6        60
## 29      160       3   2    150   3.0          17.0    13       160
## 30      120       2   1    190   0.0          15.0     9        40
## 31      140       3   2    220   3.0          21.0     7       130
## 32       90       3   0    170   3.0          18.0     2        90
## 33      100       3   0    320   1.0          20.0     3        45
## 34      120       3   1    210   5.0          14.0    12       240
## 35      110       2   0    290   0.0          22.0     3        35
## 36      110       2   1     70   1.0           9.0    15        40
## 37      110       6   0    230   1.0          16.0     3        55
## 38      120       1   2    220   0.0          12.0    12        35
## 39      120       1   2    220   1.0          12.0    11        45
## 40      100       4   2    150   2.0          12.0     6        95
## 41       50       1   0      0   0.0          13.0     0        15
## 42       50       2   0      0   1.0          10.0     0        50
## 43      100       5   2      0   2.7           1.0     1       110
```

Access all but the first row

```r
cereal[-1,]
```

```
##                     Cereal Calories Protein Fat Sodium Fiber Carbohydrates
## 2                 Cheerios      110       6   2    290   2.0          17.0
## 3               CocoaPuffs      110       1   1    180   0.0          12.0
## 4             CountChocula      110       1   1    180   0.0          12.0
## 5            GoldenGrahams      110       1   1    280   0.0          15.0
## 6         HoneyNutCheerios      110       3   1    250   1.5          11.5
## 7                      Kix      110       2   1    260   0.0          21.0
## 8              LuckyCharms      110       2   1    180   0.0          12.0
## 9       MultiGrainCheerios      100       2   1    220   2.0          15.0
## 10      OatmealRaisinCrisp      130       3   2    170   1.5          13.5
## 11           RaisinNutBran      100       3   2    140   2.5          10.5
## 12         TotalCornFlakes      110       2   1    200   0.0          21.0
## 13         TotalRaisinBran      140       3   1    190   4.0          15.0
## 14         TotalWholeGrain      100       3   1    200   3.0          16.0
## 15                    Trix      110       1   1    140   0.0          13.0
## 16                Cheaties      100       3   1    200   3.0          17.0
## 17       WheatiesHoneyGold      110       2   1    200   1.0          16.0
## 18                  AllBran       70       4   1    260   9.0           7.0
## 19              AppleJacks      110       2   0    125   1.0          11.0
## 20              CornFlakes      100       2   0    290   1.0          21.0
## 21                CornPops      110       1   0     90   1.0          13.0
## 22         CracklinOatBran      110       3   3    140   4.0          10.0
## 23                 Crispix      110       2   0    220   1.0          21.0
## 24              FrootLoops      110       2   1    125   1.0          11.0
## 25           FrostedFlakes      110       1   0    200   1.0          14.0
## 26       FrostedMiniWheats      100       3   0      0   3.0          14.0
## 27            FruitfulBran      120       3   0    240   5.0          14.0
## 28 JustRightCrunchyNuggets      110       2   1    170   1.0          17.0
## 29      MueslixCrispyBlend      160       3   2    150   3.0          17.0
## 30         NutNHoneyCrunch      120       2   1    190   0.0          15.0
## 31  NutriGrainAlmondRaisin      140       3   2    220   3.0          21.0
## 32         NutriGrainWheat       90       3   0    170   3.0          18.0
## 33               Product19      100       3   0    320   1.0          20.0
## 34              RaisinBran      120       3   1    210   5.0          14.0
## 35            RiceKrispies      110       2   0    290   0.0          22.0
## 36                  Smacks      110       2   1     70   1.0           9.0
## 37                 SpecialK      110       6   0    230   1.0          16.0
## 38              CapNCrunch      120       1   2    220   0.0          12.0
## 39          HoneyGrahamOhs      120       1   2    220   1.0          12.0
## 40                    Life      100       4   2    150   2.0          12.0
## 41              PuffedRice       50       1   0      0   0.0          13.0
## 42             PuffedWheat       50       2   0      0   1.0          10.0
## 43           QuakerOatmeal      100       5   2      0   2.7           1.0
##    Sugar Potassium
## 2      1       105
## 3     13        55
## 4     13        65
## 5      9        45
## 6     10        90
## 7      3        40
## 8     12        55
```

```
## 9       6         90
## 10     10        120
## 11      8        140
## 12      3         35
## 13     14        230
## 14      3        110
## 15     12         25
## 16      3        110
## 17      8         60
## 18      5        320
## 19     14         30
## 20      2         35
## 21     12         20
## 22      7        160
## 23      3         30
## 24     13         30
## 25     11         25
## 26      7        100
## 27     12        190
## 28      6         60
## 29     13        160
## 30      9         40
## 31      7        130
## 32      2         90
## 33      3         45
## 34     12        240
## 35      3         35
## 36     15         40
## 37      3         55
## 38     12         35
## 39     11         45
## 40      6         95
## 41      0         15
## 42      0         50
## 43      1        110
```

Access the Cereal, Protein, Fiber, and Potassium columns with all rows and save to a new dataframe

```r
cereal_pos <- cereal[,c('Cereal', 'Protein', 'Fiber', 'Potassium')]
head(cereal_pos)
```

```
##             Cereal Protein Fiber Potassium
## 1       ACCheerios       2   1.5        70
## 2         Cheerios       6   2.0       105
## 3       CocoaPuffs       1   0.0        55
## 4     CountChocula       1   0.0        65
## 5    GoldenGrahams       1   0.0        45
## 6 HoneyNutCheerios       3   1.5        90
```

Find the cereal brands for which protein is at least 3 and print those rows from the cereal_pos DataFrame

```r
cereal_pos[cereal_pos$Protein >= 3,]
```

```
##                    Cereal Protein Fiber Potassium
## 2                Cheerios       6   2.0       105
## 6         HoneyNutCheerios       3   1.5        90
## 10      OatmealRaisinCrisp       3   1.5       120
## 11           RaisinNutBran       3   2.5       140
## 13         TotalRaisinBran       3   4.0       230
## 14         TotalWholeGrain       3   3.0       110
## 16                Cheaties       3   3.0       110
## 18                 AllBran       4   9.0       320
## 22         CracklinOatBran       3   4.0       160
## 26       FrostedMiniWheats       3   3.0       100
## 27            FruitfulBran       3   5.0       190
## 29      MueslixCrispyBlend       3   3.0       160
## 31 NutriGrainAlmondRaisin       3   3.0       130
## 32         NutriGrainWheat       3   3.0        90
## 33               Product19       3   1.0        45
## 34              RaisinBran       3   5.0       240
## 37                SpecialK       6   1.0        55
## 40                    Life       4   2.0        95
## 43           QuakerOatmeal       5   2.7       110
```

```r
cereal_pos[cereal$Protein >= 3,]
```

```
##                    Cereal Protein Fiber Potassium
## 2                Cheerios       6   2.0       105
## 6         HoneyNutCheerios       3   1.5        90
## 10      OatmealRaisinCrisp       3   1.5       120
## 11           RaisinNutBran       3   2.5       140
## 13         TotalRaisinBran       3   4.0       230
## 14         TotalWholeGrain       3   3.0       110
## 16                Cheaties       3   3.0       110
## 18                 AllBran       4   9.0       320
## 22         CracklinOatBran       3   4.0       160
## 26       FrostedMiniWheats       3   3.0       100
## 27            FruitfulBran       3   5.0       190
## 29      MueslixCrispyBlend       3   3.0       160
## 31 NutriGrainAlmondRaisin       3   3.0       130
## 32         NutriGrainWheat       3   3.0        90
## 33               Product19       3   1.0        45
## 34              RaisinBran       3   5.0       240
## 37                SpecialK       6   1.0        55
## 40                    Life       4   2.0        95
## 43           QuakerOatmeal       5   2.7       110
```

dim() Function

DataFrame equivalent of length()

```r
dim(cereal) # output is in rows, columns
```

```
## [1] 43  9
```

```r
nrow(cereal)
```

```
## [1] 43
```

```r
ncol(cereal)
```

```
## [1] 9
```

```r
head(cereal)
```

```
##             Cereal Calories Protein Fat Sodium Fiber Carbohydrates Sugar
## 1       ACCheerios      110       2   2    180   1.5          10.5    10
## 2         Cheerios      110       6   2    290   2.0          17.0     1
## 3       CocoaPuffs      110       1   1    180   0.0          12.0    13
## 4     CountChocula      110       1   1    180   0.0          12.0    13
## 5    GoldenGrahams      110       1   1    280   0.0          15.0     9
## 6 HoneyNutCheerios      110       3   1    250   1.5          11.5    10
##   Potassium
## 1        70
## 2       105
## 3        55
## 4        65
## 5        45
## 6        90
```

```r
tail(cereal)
```

```
##            Cereal Calories Protein Fat Sodium Fiber Carbohydrates Sugar
## 38     CapNCrunch      120       1   2    220   0.0            12    12
## 39 HoneyGrahamOhs      120       1   2    220   1.0            12    11
## 40           Life      100       4   2    150   2.0            12     6
## 41     PuffedRice       50       1   0      0   0.0            13     0
## 42    PuffedWheat       50       2   0      0   1.0            10     0
## 43  QuakerOatmeal      100       5   2      0   2.7             1     1
##    Potassium
## 38        35
## 39        45
## 40        95
## 41        15
## 42        50
## 43       110
```

```r
summary(cereal)
```

```
##         Cereal      Calories        Protein           Fat
##  ACCheerios: 1   Min.   : 50.0   Min.   :1.000   Min.   :0.0000
##  AllBran   : 1   1st Qu.:100.0   1st Qu.:2.000   1st Qu.:0.0000
##  AppleJacks: 1   Median :110.0   Median :2.000   Median :1.0000
##  CapNCrunch: 1   Mean   :107.9   Mean   :2.465   Mean   :0.9767
##  Cheaties  : 1   3rd Qu.:110.0   3rd Qu.:3.000   3rd Qu.:1.5000
##  Cheerios  : 1   Max.   :160.0   Max.   :6.000   Max.   :3.0000
##  (Other)   :37
##      Sodium          Fiber       Carbohydrates       Sugar
##  Min.   :  0.0   Min.   :0.000   Min.   : 1.00   Min.   : 0.000
##  1st Qu.:145.0   1st Qu.:0.500   1st Qu.:12.00   1st Qu.: 3.000
##  Median :190.0   Median :1.000   Median :14.00   Median : 8.000
##  Mean   :180.5   Mean   :1.714   Mean   :14.26   Mean   : 7.605
```

```
##  3rd Qu.:220.0   3rd Qu.:2.850   3rd Qu.:17.00   3rd Qu.:12.000
##  Max.   :320.0   Max.   :9.000   Max.   :22.00   Max.   :15.000
##
##    Potassium
##  Min.   : 15.00
##  1st Qu.: 37.50
##  Median : 60.00
##  Mean   : 84.42
##  3rd Qu.:110.00
##  Max.   :320.00
##
```

```r
str(cereal)
```

```
## 'data.frame':    43 obs. of  9 variables:
##  $ Cereal       : Factor w/ 43 levels "ACCheerios","AllBran",..: 1 6 7 10 17 19 21 23 25 29 ...
##  $ Calories     : int  110 110 110 110 110 110 110 110 100 130 ...
##  $ Protein      : int  2 6 1 1 1 3 2 2 2 3 ...
##  $ Fat          : int  2 2 1 1 1 1 1 1 1 2 ...
##  $ Sodium       : int  180 290 180 180 280 250 260 180 220 170 ...
##  $ Fiber        : num  1.5 2 0 0 0 1.5 0 0 2 1.5 ...
##  $ Carbohydrates: num  10.5 17 12 12 15 11.5 21 12 15 13.5 ...
##  $ Sugar        : int  10 1 13 13 9 10 3 12 6 10 ...
##  $ Potassium    : int  70 105 55 65 45 90 40 55 90 120 ...
```

```r
levels(cereal)
```

```
## NULL
```

```r
levels(cereal$Cereal)
```

```
##  [1] "ACCheerios"              "AllBran"
##  [3] "AppleJacks"              "CapNCrunch"
##  [5] "Cheaties"                "Cheerios"
##  [7] "CocoaPuffs"              "CornFlakes"
##  [9] "CornPops"                "CountChocula"
## [11] "CracklinOatBran"         "Crispix"
## [13] "FrootLoops"              "FrostedFlakes"
## [15] "FrostedMiniWheats"       "FruitfulBran"
## [17] "GoldenGrahams"           "HoneyGrahamOhs"
## [19] "HoneyNutCheerios"        "JustRightCrunchyNuggets"
## [21] "Kix"                     "Life"
## [23] "LuckyCharms"             "MueslixCrispyBlend"
## [25] "MultiGrainCheerios"      "NutNHoneyCrunch"
## [27] "NutriGrainAlmondRaisin"  "NutriGrainWheat"
## [29] "OatmealRaisinCrisp"      "Product19"
## [31] "PuffedRice"              "PuffedWheat"
## [33] "QuakerOatmeal"           "RaisinBran"
## [35] "RaisinNutBran"           "RiceKrispies"
## [37] "Smacks"                  "SpecialK"
## [39] "TotalCornFlakes"         "TotalRaisinBran"
## [41] "TotalWholeGrain"         "Trix"
## [43] "WheatiesHoneyGold"
```

```r
mean(cereal$Calories)
```

```
## [1] 107.907
```

```r
sd(cereal$Calories)
```

```
## [1] 18.96841
```

```r
median(cereal$Calories)
```

```
## [1] 110
```

```r
quantile(cereal$Calories)
```

```
##   0%  25%  50%  75% 100%
##   50  100  110  110  160
```

```r
min(cereal$Calories)
```

```
## [1] 50
```

```r
max(cereal$Calories)
```

```
## [1] 160
```

```r
summary(cereal$Calories)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    50.0   100.0   110.0   107.9   110.0   160.0
```

Note: levels() can be used to reassign names of elements in the same way names can be used to reassign names of columns